



Instituto Politécnico Nacional



**UPIITA (Unidad Profesional
Interdisciplinaria en Ingeniería y
Tecnologías Avanzadas)**



Base de Datos Distribuidos

Profesor: De la cruz Sosa Carlos

Asignación: Practica 4

Alumno:

-Gutiérrez Flores José Julián

-Muñoz Reyes Citlali

Práctica 4

Instrucciones:

la práctica 4 es por equipos y consiste en codificar, probar y analizar cada uno de los problemas de concurrencia descritos en el libro <https://www.dropbox.com/scl/fi/2mfn16p5jzuhmrpk1h167/Microsoft-SQL-Server-2000-Programming-by-Example.pdf?rlkey=m6d30w8u8vh6g1qs763131d8t&st=4q5tqg4b&dl=0>, excepto el problema de actualización perdida. Consideren la BD de Adventureworks para la codificación.

Deben subir a su repositorio de GitHub:

- PDF con el nombre de practica_4, que incluya una descripción de cada problema, el código ejemplo y conclusiones de lo que analizaron en cada escenario.

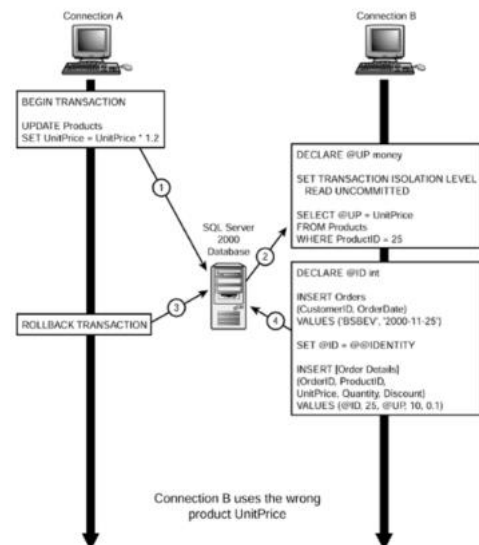
Uncommitted Dependency (Dirty Read) → Lectura Sucia

1. Descripción del problema

La Lectura sucia en bases de datos son un fenómeno de concurrencia donde una transacción lee datos modificados por otra transacción simultánea que aún no han sido confirmados (commit). Si la primera transacción revierte (rollback) sus cambios, la segunda habrá leído datos incorrectos o inexistentes, provocando inconsistencias en la información.

Normalmente se presenta cuando se lee datos sin usar bloqueos, ya que se obtiene resultados inesperados, debido a que otras transiciones pueden modificar los datos temporalmente.

En este caso tenemos que el proceso A hace una modificación en la tabla de Sales.Customer y el proceso B está consultándolo en el momento en que se está actualizando, pero este dato al poco momento queda igual que como estaba antes, porque el proceso A aplica ROLLBACK TRANSACTION, sin embargo, el proceso B ya lo recibió.



	CustomerID	PersonID	StoreID	TerritoryID	AccountNumber	rowguid	ModifiedDate
1	1	NULL	934	1	AW00000001	3F5AE95E-B87D-4AED-95B4-C3797AFCB74F	2014-09-12 11:15:07.263
2	2	NULL	1028	1	AW00000002	E552F857-A9AF-4A7D-A645-C429D6E02491	2014-09-12 11:15:07.263
3	3	NULL	642	4	AW00000003	130774B1-DB21-4EF3-98C8-C104BCD6ED6D	2014-09-12 11:15:07.263
4	4	NULL	932	4	AW00000004	FF862851-1DAA-4044-BE7C-3E85583C054D	2014-09-12 11:15:07.263
5	5	NULL	1026	4	AW00000005	83905BDC-6F5E-4F71-B162-C98DA069F38A	2014-09-12 11:15:07.263
6	6	NULL	644	4	AW00000006	1A92DF88-BFA2-467D-BD54-FCB9E647FDD7	2014-09-12 11:15:07.263
7	7	NULL	930	1	AW00000007	03E9273E-B193-448E-9823-FE0C44AEE078	2014-09-12 11:15:07.263
8	8	NULL	1024	5	AW00000008	801368B1-4323-4BFA-8BEA-5B5B1E4BD4A0	2014-09-12 11:15:07.263
9	9	NULL	620	5	AW00000009	B900B87F-23C3-481D-80DA-C49A5BD6F772	2014-09-12 11:15:07.263
10	10	NULL	928	6	AW00000010	CDB6698D-2FF1-4FBA-8F22-60AD1D11DABD	2014-09-12 11:15:07.263
11	11	NULL	1022	6	AW00000011	750F3495-59C4-48A0-80E1-E37EC60E77D9	2014-09-12 11:15:07.263
12	12	NULL	622	6	AW00000012	947BCAF1-1F32-44F3-B9C3-0011F95FBE54	2014-09-12 11:15:07.263
13	13	NULL	434	7	AW00000013	B0FA5854-2511-439B-A7AC-50C9C460B175	2014-09-12 11:15:07.263
14	14	NULL	1020	8	AW00000014	2F96BEDC-723D-468F-834B-B2B8AE79C849	2014-09-12 11:15:07.263
15	15	NULL	624	9	AW00000015	0340737B-D4FA-4795-93AA-CAEB8371BCF9	2014-09-12 11:15:07.263
16	16	NULL	432	10	AW00000016	C9381589-D31C-4EFE-8978-8D3449EB1F0F	2014-09-12 11:15:07.263

Imagen1. Dato a modificar

2. Código

Proceso A

```
/*Secuencia de pasos*/

-- Proceso A
USE AdventureWorks2022;
GO
-- 1. Transacción abierta
BEGIN TRANSACTION TerritoryID;

-- 1.1. Modificamos un registro (BusinessEntityID = 13)
UPDATE Sales.Customer
SET TerritoryID = '3'
WHERE CustomerID = 13;

PRINT 'Proceso A: El registro se actualizo.
Esperando antes de hacer COMMIT/ROLLBACK...';

-- 1.2 Pausamos la ejecución: 20 segundos
WAITFOR DELAY '00:00:20';

-- 3. Simulamos una cancelación o error,
revirtiendo el cambio
ROLLBACK TRANSACTION;
PRINT 'Proceso A: Transacción revertida
(ROLLBACK ejecutado).';
GO
```

Proceso B

```
USE AdventureWorks2022;
GO

-- 2.Nivel de aislamiento:
SET TRANSACTION ISOLATION LEVEL READ
UNCOMMITTED

PRINT 'Proceso B: Leyendo datos no
confirmados...';
-- 2.1 Consultamos el mismo registro que el
Proceso A está modificando
SELECT CustomerID, StoreID, TerritoryID
FROM Sales.Customer
WHERE CustomerID = 13;
GO

--4.Cambio de dato
UPDATE Sales.Customer
SET TerritoryID = '10'
WHERE CustomerID = 13;
```

2.2 Reproducción del problema

proceso_B.sq...JK4P\hp (51)* proceso_A.sq...jecutando...*

```
39
40
41
42 /*Secuencia de pasos*/
43
44 -- Proceso A
45 USE AdventureWorks2022;
46 GO
47 -- 1.Transacción abierta
48 BEGIN TRANSACTION TerritoryID;
49
50 -- 1.1. Modificamos un registro (BusinessEntityID = 13)
51 UPDATE Sales.Customer
52 SET TerritoryID = '3'
53 WHERE CustomerID = 13;
54
55 PRINT 'Proceso A: El registro se actualizo. Esperando antes de
56
57 -- 1.2 Pausamos la ejecución: 20 segundos
58 WAITFOR DELAY '00:00:20';
59
60 -- 3. Simulamos una cancelación o error, revirtiendo el cambio
61 ROLLBACK TRANSACTION;
62 PRINT 'Proceso A: Transacción revertida (ROLLBACK ejecutado).
63
64
65
66 /*Verificacion*/
67 SELECT* FROM Sales.Customer
```

100 % No se encontraron problemas.

Resultados Mensajes

Ejecutando consulta...

Se llevó a cabo el proceso A

El proceso B leyó el dato modificado

proceso_B.s...4P\hp (51)* proceso_A.sq...JK4P\hp (54)*

```
1 /* proceso B */
2
3
4 USE AdventureWorks2022;
5 GO
6
7 -- 2.Nivel de aislamiento:
8 SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED
9
10 PRINT 'Proceso B: Leyendo datos no confirmados...';
11 -- 2.1 Consultamos el mismo registro que el Proceso A e
12 SELECT CustomerID, StoreID, TerritoryID
13 FROM Sales.Customer
14 WHERE CustomerID = 13;
15 GO
16
17 --4.Cambio de dato
18 UPDATE Sales.Customer
19 SET TerritoryID = '10'
20 WHERE CustomerID = 13;
21
22
23
24
25
```

100 % No se encontraron problemas.

Resultados Mensajes

CustomerID	StoreID	TerritoryID
13	434	3

100 % No se encontraron problemas.

Resultados Mensajes

Proceso B: Leyendo datos no confirmados...

(1 fila afectada)

(1 fila afectada)

Hora de finalización: 2026-09-24T03:19:11.3094443-06:00

100 % ✔ No se encontraron problemas.

Mensajes

(1 fila afectada)

Proceso A: El registro se actualizo. Esperando antes de hacer COMMIT/ROLLBACK...
Proceso A: Transacción revertida (ROLLBACK ejecutado).

Hora de finalización: 2026-09-24T03:19:11.3709185-06:00

El proceso A revirtió su transacción, sin embargo, el proceso B ya lo había leído e hizo la su modificación.

100 % ✔ No se encontraron problemas.

Resultados

Mensajes

	CustomerID	PersonID	StoreID	TerritoryID	AccountNumber	rowguid	ModifiedDate
5	5	NULL	1026	4	AW00000005	83905BDC-6F5E-4F71-B162-C98DA069F38A	2014-09-12 11:15:07.263
6	6	NULL	644	4	AW00000006	1A92DF88-BFA2-467D-BD54-FCB9E647FDD7	2014-09-12 11:15:07.263
7	7	NULL	930	1	AW00000007	03E9273E-B193-448E-9823-FE0C44AEED78	2014-09-12 11:15:07.263
8	8	NULL	1024	5	AW00000008	801368B1-4323-4BFA-8BEA-5B5B1E4BD4A0	2014-09-12 11:15:07.263
9	9	NULL	620	5	AW00000009	B900BB7F-23C3-481D-80DA-C49A5BD6F772	2014-09-12 11:15:07.263
10	10	NULL	928	6	AW00000010	CDB6698D-2FF1-4FBA-8F22-60AD1D11DABD	2014-09-12 11:15:07.263
11	11	NULL	1022	6	AW00000011	750F3495-59C4-48A0-80E1-E37EC60E77D9	2014-09-12 11:15:07.263
12	12	NULL	622	6	AW00000012	947BCAF1-1F32-44F3-B9C3-0011F95FBE54	2014-09-12 11:15:07.263
13	13	NULL	434	10	AW00000013	B0FA5854-2511-439B-A7AC-50C9C460B175	2014-09-12 11:15:07.263

3. Conclusiones

-El permitir el problema de lectura sucia, puede exponer a la confiabilidad de la base de datos, debido a que, al tomar decisiones operativas o financieras basadas en datos volátiles, causa que los datos lleguen a fallar por la modificación o cancelación; alterando cálculos o lógica ejecutada por datos leídos erróneamente.

-En esta demostración se tuvo que trabajar con un nivel bajo de aislamiento (READ UNCOMMITTED), ya que el entorno tiene solución ante estos problemas de concurrencia, en dado caso de que se quisiera evitar se podría ocupar los bloqueos o el nivel de aislamiento READ COMMITTED.

Inconsistent Analysis (Nonrepeatable Read) -> lectura no repetible

1. Descripción del problema

Ocurren cuando una transacción lee la misma fila dos veces, pero obtiene valores diferentes en cada lectura debido a que otra transacción ha modificado y confirmado esos datos entre las dos lecturas. Este fenómeno se produce porque, aunque la primera transacción ya leyó los datos, otra transacción concurrente puede haber actualizado el valor de la fila y confirmado esos cambios.

Esto también se puede considerar como un análisis inconsistente, porque cada vez que se leen los datos, estos son diferentes.

El problema que observaremos está enfocado en la tabla Person.Person, donde se hará un conteo de cuantos nombres empiezan con la letra A en el proceso A, siendo esta consulta repetitiva, mientras el proceso B también se lleva a acabo y va a agregar y modificar datos, causando que se tengan datos inconsistentes.

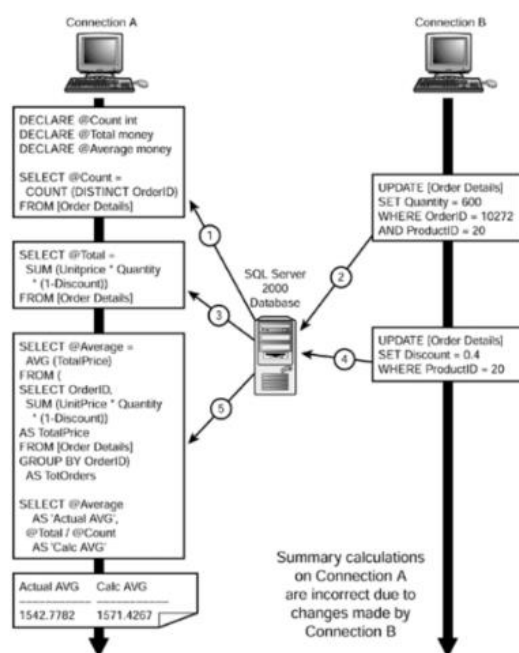


Tabla a manipular

	BusinessEntityID	PersonType	NameStyle	Title	FirstName	MiddleName	LastName	Suffix	EmailPromotion	AdditionalContactInfo	Demographics	rowguid	Mod
1	1	EM	0	NULL	Ken	J	Sánchez	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso...	92C4279F-1207-48A3-8448-4636514EB7E2	2006
2	2	EM	0	NULL	Terri	Lee	Duffy	NULL	1	NULL	<IndividualSurvey xmlns="http://schemas.microso...	D8763459-8A48-47CC-AFF7-C9079AF79033	2006
3	3	EM	0	NULL	Roberto	NULL	Tamburello	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso...	E1A2555E-0828-434B-A338-6F38136A37DE	2006
4	4	EM	0	NULL	Rob	NULL	Walters	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso...	F2D7CE06-38B3-4357-8058-F4B6B71C01F1	2006
5	5	EM	0	Ms.	Gail	A	Erickson	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso...	F3A3F6B4-AE3B-430C-A754-9F2231BA6FEF	2006
6	6	EM	0	Mr.	Josef	H	Goldberg	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso...	0DEA28FD-EFFE-482A-AFD3-B7E9F199D56F	2011
7	7	EM	0	NULL	Dylan	A	Miller	NULL	2	NULL	<IndividualSurvey xmlns="http://schemas.microso...	C45E9A89-01BE-4B76-B215-820C8368181A	2006
8	8	EM	0	NULL	Diane	L	Margheim	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso...	A948E590-4A56-45A9-BC9A-160A1C09D990	2006
9	9	EM	0	NULL	Gigi	N	Matthew	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso...	5FC28C0E-6D36-4252-8846-05CAA081F6C5	2006
10	10	EM	0	NULL	Michael	NULL	Raheem	NULL	2	NULL	<IndividualSurvey xmlns="http://schemas.microso...	CAC740E-75B2-420C-BD48-E3C9C0609604	2006
11	11	EM	0	NULL	Ovidiu	V	Cracium	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso...	D2CC2577-4F6B-4408-BD8C-747337FE5645	2011
12	12	EM	0	NULL	Thierry	B	D'Hers	NULL	2	NULL	<IndividualSurvey xmlns="http://schemas.microso...	FA263C7F-600D-4E59-8DCD-9878F3530F5F	2006
13	13	EM	0	Ms.	Janice	M	Galvin	NULL	2	NULL	<IndividualSurvey xmlns="http://schemas.microso...	34E999E0-7042-4DC1-A2FD-BDA290FF0E07	2011
14	14	EM	0	NULL	Michael	I	Sullivan	NULL	2	NULL	<IndividualSurvey xmlns="http://schemas.microso...	9A751DE-5CAF-4700-AB07-CC81102B8696	2011
15	15	EM	0	NULL	Sharon	B	Salavania	NULL	2	NULL	<IndividualSurvey xmlns="http://schemas.microso...	BEB463CB-13F1-4B76-A3DE-FE9AC283A94C	2011
16	16	EM	0	NULL	David	M	Bradley	NULL	1	NULL	<IndividualSurvey xmlns="http://schemas.microso...	ZCC8BA72-5D8B-497F-B941-775517993638	2006

2. Código

```
--Proceso A
select *from Person.Person

USE AdventureWorks2022;
GO

--Nivel de aislamiento
SET TRANSACTION ISOLATION LEVEL READ
COMMITTED;

BEGIN TRANSACTION;

DECLARE @TotalPersonas INT;

-- Paso 1: Conteo inicial de personas cuyo
FirstName empieza con 'A'
SELECT @TotalPersonas = COUNT(*)
FROM Person.Person
WHERE FirstName LIKE 'A%';

PRINT 'Proceso A. 1 - Total inicial de
personas con A: ' + CAST(@TotalPersonas AS
VARCHAR);

-- espera
WAITFOR DELAY '00:00:15';

SELECT Title, COUNT(*) AS Cantidad
FROM Person.Person
WHERE FirstName LIKE 'A%'
GROUP BY Title;

WAITFOR DELAY '00:00:15';

-----
DECLARE @TotalFinal INT;
SELECT @TotalFinal = COUNT(*)
FROM Person.Person
WHERE FirstName LIKE 'A%';

PRINT 'Paso 5 - Total final de personas con
A: ' + CAST(@TotalFinal AS VARCHAR);

COMMIT TRANSACTION;
```

```
--Proceso B
USE AdventureWorks2022;
GO

BEGIN TRANSACTION;

-- Paso 2: Cambiar el FirstName de una
persona que no empezaba con 'A' para que
ahora empiece con 'A'
-- O renombrar a alguien que empezaba con
'A' para que empiece con otra letra
UPDATE Person.Person
SET FirstName = 'Alexander'
WHERE BusinessEntityID = 1; -- Originalmente
es 'Ken'

-- Paso 4: Cambiar el título (Title) de
varias personas
UPDATE Person.Person
SET Title = 'Dr.'
WHERE FirstName LIKE 'A%' AND PersonType =
'EM';

COMMIT TRANSACTION;
```

2.2 Reproducción del problema

```
procesoBno_r_JK4P\hp (56)  proceso_Ano_r_JK4P\hp (56)
1  --Proceso A
2  select *from Person.Person
3
4
5  USE AdventureWorks2022;
6  GO
7  --Nivel de aislamiento
8  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
9
10 BEGIN TRANSACTION;
11
12 DECLARE @TotalPersonas INT;
13
14 -- Paso 1: Conteo inicial de personas cuyo FirstName empieza con 'A'
15 SELECT @TotalPersonas = COUNT(*)
16 FROM Person.Person
17 WHERE FirstName LIKE 'A%';
18
19 PRINT 'Proceso A. 1 - Total inicial de personas con A: ' + CAST(@TotalPersonas AS VARCHAR);
20
21 -- espera
22 WAITFOR DELAY '00:00:15';
23
24 SELECT Title, COUNT(*) AS Cantidad
25 FROM Person.Person
26
```

Se llevó a cabo el proceso A

```
procesoBno_r_JK4P\hp (56)  proceso_Ano_r_JK4P\hp (54)
1  --Proceso B
2  USE AdventureWorks2022;
3  GO
4
5  BEGIN TRANSACTION;
6
7  -- Paso 2: Cambiar el FirstName de una persona que no empezaba con 'A'
8  -- 0 renombrar a alguien que empezaba con 'A' para que empiece con 'A'
9  UPDATE Person.Person
10 SET FirstName = 'Alexander'
11 WHERE BusinessEntityID = 1; -- Originalmente es 'Ken'
12
13 -- Paso 4: Cambiar el titulo (Title) de varias personas
14 UPDATE Person.Person
15 SET Title = 'Dr.'
16 WHERE FirstName LIKE 'A%' AND PersonType = 'EM';
17
18 COMMIT TRANSACTION;
```

El proceso B leyó el dato modificado

```
procesoBno_r_JK4P\hp (56)  proceso_Ano_r_JK4P\hp (54)*
1  --Proceso A
2  select *from Person.Person
3
4
5  USE AdventureWorks2022;
6  GO
7  --Nivel de aislamiento
8  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
9
10 BEGIN TRANSACTION;
11
12 DECLARE @TotalPersonas INT;
13
14 -- Paso 1: Conteo inicial de personas cuyo FirstName empieza con 'A'
15 SELECT @TotalPersonas = COUNT(*)
16 FROM Person.Person
17 WHERE FirstName LIKE 'A%';
18
19 PRINT 'Proceso A. 1 - Total inicial de personas con A: ' + CAST(@TotalPersonas AS VARCHAR);
20
21 -- espera
22 WAITFOR DELAY '00:00:15';
23
24 SELECT Title, COUNT(*) AS Cantidad
25 FROM Person.Person
26
```

Tras repetir una vez más la consulta se encontró un nuevo dato

100 % No se encontraron problemas.		
Resultados Mensajes		
	Title	Cantidad
1	Sr.	3
2	Dr.	16
3	Mrs.	1
4	NULL	1931
5	Ms.	31
6	Ms	1
7	Mr.	32

3. Conclusiones:

-Este problema se puede prevenir minimizando las lecturas repetidas e intentando convertirlas en operaciones atómicas, o utilizando el nivel de aislamiento **Repeatable read**; ya que al utilizarlo los datos recuperados se bloquean hasta que finaliza la transacción, lo que impide que otras conexiones realicen cambios en los datos.

-En esta replicación del problema tuvimos que ocupar “**SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;**” debido a que este nivel de acepta lo que son las lecturas inconsistentes o no repetibles,

Phantom Reads -> Lectura fantasma

1. Descripción del problema

Phantom Reads (Lecturas Fantasma) es un problema de concurrencia que ocurre cuando una transacción ejecuta la misma consulta con una condición de rango (WHERE) dos veces, y entre ambas ejecuciones otra transacción concurrente inserta o elimina filas que cumplen esa condición. Como resultado, la segunda lectura devuelve un conjunto de filas diferente al de la primera, a pesar de que ninguna fila previamente leída fue modificada.

La diferencia clave respecto al problema de *Nonrepeatable Read* es que ahí cambia el **valor** de una fila ya leída, mientras que en *Phantom Reads* cambia el **número de filas** que cumplen el criterio de búsqueda.

Este fenómeno ocurre bajo los niveles de aislamiento READ COMMITTED y REPEATABLE READ, pero es evitado por el nivel SERIALIZABLE, ya que este último bloquea no solo las filas existentes sino también el rango completo de valores consultado (mediante *key-range locking*), impidiendo inserciones concurrentes dentro de ese rango.

Se utilizó la base de datos **AdventureWorks2022**, específicamente la tabla Production.Product, filtrando por la columna ListPrice en el rango de 500 a 1000.

2. Código

2.1 Limpieza previa (antes de cada prueba)

```
DELETE FROM Production.Product WHERE ProductNumber = 'FANT-001';
```

2.2 Reproducción del problema (nivel READ COMMITTED)

Sesión 1 – Transacción lectora:

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
BEGIN TRANSACTION;
```

```
SELECT ProductID, Name, ListPrice  
FROM Production.Product  
WHERE ListPrice BETWEEN 500 AND 1000  
ORDER BY ProductID DESC;
```

SQLQuery7.sql...22 (sa (54))* X SQLQuery8.sql...ela (sa (57))*

```
1 SELECT @@TRANCOUNT;  
2 USE AdventureWorks2022  
3 DELETE FROM Production.Product WHERE ProductNumber = 'FANT-001';  
4  
5 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
6 BEGIN TRANSACTION;  
7  
8 SELECT ProductID, Name, ListPrice  
9 FROM Production.Product  
10 WHERE ListPrice BETWEEN 500 AND 1000  
11 ORDER BY ProductID DESC;
```

97 % No se encontraron problemas. Línea: 5

Resultados Mensajes

	ProductID	Name	ListPrice
1	999	Road-750 Black, 52	539.99
2	998	Road-750 Black, 48	539.99
3	997	Road-750 Black, 44	539.99
4	993	Mountain-500 Black, 52	539.99
5	992	Mountain-500 Black, 48	539.99
6	991	Mountain-500 Black, 44	539.99
7	990	Mountain-500 Black, 42	539.99
8	989	Mountain-500 Black, 40	539.99
9	988	Mountain-500 Silver, 52	564.99
10	987	Mountain-500 Silver, 48	564.99
11	986	Mountain-500 Silver, 44	564.99

Sesión 2 – Transacción que inserta la fila fantasma:

```
BEGIN TRANSACTION;
```

```
INSERT INTO Production.Product  
(Name, ProductNumber, MakeFlag, FinishedGoodsFlag, SafetyStockLevel,  
ReorderPoint, StandardCost, ListPrice, DaysToManufacture,  
SellStartDate, rowguid, ModifiedDate)  
VALUES  
( 'Producto Fantasma', 'FANT-001', 0, 0, 100, 75, 600.00, 750.00, 0,  
GETDATE(), NEWID(), GETDATE());
```

```
COMMIT TRANSACTION;
```

The screenshot displays the SQL Server Enterprise Manager interface. At the top, there are two tabs: 'SQLQuery7.sql...022 (sa (54))*' and 'SQLQuery8.sql...22 (sa (57))*'. The active window shows a SQL script with the following lines:

```
1 SELECT @@TRANCOUNT;  
2 USE AdventureWorks2022  
3  
4 BEGIN TRANSACTION;  
5  
6 INSERT INTO Production.Product  
7 (Name, ProductNumber, MakeFlag, FinishedGoodsFlag, SafetyStockLevel,  
8 ReorderPoint, StandardCost, ListPrice, DaysToManufacture,  
9 SellStartDate, rowguid, ModifiedDate)  
10 VALUES  
11 ('Producto Fantasma', 'FANT-001', 0, 0, 100, 75, 600.00, 750.00, 0,  
12 GETDATE(), NEWID(), GETDATE());  
13  
14 COMMIT TRANSACTION;
```

Below the script, a status bar indicates '97 %' and 'No se encontraron problemas.' with a green checkmark icon. To the right, it says 'Línea: 4, Caráct...'. Below this is a 'Mensajes' (Messages) window showing the execution result:

```
(1 fila afectada)  
  
Hora de finalización: 2026-09-23T23:37:01.5286100-06:00
```

Sesión 1 – Segunda lectura (misma transacción):

```
SELECT ProductID, Name, ListPrice  
FROM Production.Product  
WHERE ListPrice BETWEEN 500 AND 1000  
ORDER BY ProductID DESC;
```

```
COMMIT TRANSACTION;
```

SQLQuery7.sql...22 (sa (54))*

SQLQuery8.sql...022 (sa (57))*

```

4
5     SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
6     BEGIN TRANSACTION;
7
8     SELECT ProductID, Name, ListPrice
9     FROM Production.Product
10    WHERE ListPrice BETWEEN 500 AND 1000
11    ORDER BY ProductID DESC;
12
13
14    SELECT ProductID, Name, ListPrice
15    FROM Production.Product
16    WHERE ListPrice BETWEEN 500 AND 1000
17    ORDER BY ProductID DESC;

```

97 %

No se encontraron problemas.

Línea: 14, Car

Resultados

Mensajes

	ProductID	Name	ListPrice
1	1020	Producto Fantasma	750.00
2	999	Road-750 Black, 52	539.99
3	998	Road-750 Black, 48	539.99
4	997	Road-750 Black, 44	539.99
5	993	Mountain-500 Black, 52	539.99
6	992	Mountain-500 Black, 48	539.99
7	991	Mountain-500 Black, 44	539.99
8	990	Mountain-500 Black, 42	539.99
9	989	Mountain-500 Black, 40	539.99
10	988	Mountain-500 Silver, 52	564.99
11	987	Mountain-500 Silver, 48	564.99

2.3 Solución del problema (nivel SERIALIZABLE)

Sesión 1 – Transacción lectora con SERIALIZABLE:

```

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN TRANSACTION;

```

```

SELECT ProductID, Name, ListPrice
FROM Production.Product
WHERE ListPrice BETWEEN 500 AND 1000
ORDER BY ProductID DESC;

```

SQLQuery7.sql...22 (sa (54))* x SQLQuery8.sql...022 (sa (57))*

```

24
25
26 DELETE FROM Production.Product WHERE ProductNumber = 'FANT-001';
27
28
29
30 SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
31 BEGIN TRANSACTION;
32
33 SELECT ProductID, Name, ListPrice
34 FROM Production.Product
35 WHERE ListPrice BETWEEN 500 AND 1000
36 ORDER BY ProductID DESC;

```

97 % No se encontraron problemas. Línea: 30, C

Resultados Mensajes

	ProductID	Name	ListPrice
1	999	Road-750 Black, 52	539.99
2	998	Road-750 Black, 48	539.99
3	997	Road-750 Black, 44	539.99
4	993	Mountain-500 Black, 52	539.99
5	992	Mountain-500 Black, 48	539.99
6	991	Mountain-500 Black, 44	539.99
7	990	Mountain-500 Black, 42	539.99
8	989	Mountain-500 Black, 40	539.99
9	988	Mountain-500 Silver, 52	564.99
10	987	Mountain-500 Silver, 48	564.99
11	986	Mountain-500 Silver, 44	564.99

Sesión 2 – Intento de INSERT (bloqueado):

```

BEGIN TRANSACTION;
INSERT INTO Production.Product
(Name, ProductNumber, MakeFlag, FinishedGoodsFlag, SafetyStockLevel,
ReorderPoint, StandardCost, ListPrice, DaysToManufacture,
SellStartDate, rowguid, ModifiedDate)
VALUES
('Producto Fantasma', 'FANT-001', 0, 0, 100, 75, 600.00, 750.00, 0,
GETDATE(), NEWID(), GETDATE());

COMMIT TRANSACTION;

```

Ejecutar

SQLQuery7.sql...022 (sa (54))* SQLQuery8.s...jecutando...*

```

19
20 BEGIN TRANSACTION;
21
22 INSERT INTO Production.Product
23 (Name, ProductNumber, MakeFlag, FinishedGoodsFlag, SafetyStockLevel,
24 ReorderPoint, StandardCost, ListPrice, DaysToManufacture,
25 SellStartDate, rowguid, ModifiedDate)
26 VALUES
27 ('Producto Fantasma', 'FANT-001', 0, 0, 100, 75, 600.00, 750.00, 0,
28 GETDATE(), NEWID(), GETDATE());
29
30 COMMIT TRANSACTION;

```

97 % No se encontraron problemas. Línea: 20, Car

Resultados Mensajes

Sesión 1 – Segunda lectura (confirma que no hay fantasma) y cierre:

```
SELECT ProductID, Name, ListPrice
FROM Production.Product
WHERE ListPrice BETWEEN 500 AND 1000
ORDER BY ProductID DESC;
```

```
COMMIT TRANSACTION;
```

The screenshot shows the SQL Server Enterprise Manager interface. At the top, there are two query windows: 'SQLQuery7.sql...22 (sa (54))*' and 'SQLQuery8.sql...022 (sa (57))*'. The first window contains the SQL query:

```
SELECT ProductID, Name, ListPrice
FROM Production.Product
WHERE ListPrice BETWEEN 500 AND 1000
ORDER BY ProductID DESC;
COMMIT TRANSACTION;
```

 Below the query window, the 'Results' tab is active, displaying a table with 11 rows. The columns are 'ProductID', 'Name', and 'ListPrice'. The first row is highlighted. Below the table, a status bar indicates 'Consulta ejecutada correctamente.' (Query executed successfully).

	ProductID	Name	ListPrice
1	999	Road-750 Black, 52	539.99
2	998	Road-750 Black, 48	539.99
3	997	Road-750 Black, 44	539.99
4	993	Mountain-500 Black, 52	539.99
5	992	Mountain-500 Black, 48	539.99
6	991	Mountain-500 Black, 44	539.99
7	990	Mountain-500 Black, 42	539.99
8	989	Mountain-500 Black, 40	539.99
9	988	Mountain-500 Silver, 52	564.99
10	987	Mountain-500 Silver, 48	564.99
11	986	Mountain-500 Silver, 44	564.99

Conclusiones

- Con el nivel de aislamiento READ COMMITTED, el fenómeno de Phantom Read se reprodujo exitosamente: la segunda lectura dentro de la misma transacción mostró una fila nueva ("Producto Fantasma") que no existía en la primera lectura, a pesar de estar dentro de la misma transacción.
- Con el nivel de aislamiento SERIALIZABLE, el motor de SQL Server bloqueó el rango de valores consultado (ListPrice BETWEEN 500 AND 1000) mediante *key-range locks*, impidiendo que la Sesión 2 insertara una fila dentro de ese rango hasta que la Sesión 1 completara su transacción.
- El uso de SERIALIZABLE garantiza consistencia total de lectura, pero tiene un costo de concurrencia: otras transacciones que intenten modificar datos dentro del rango bloqueado deben esperar, lo cual puede reducir el rendimiento en sistemas con alta concurrencia.

- Se recomienda usar SERIALIZABLE solo cuando la consistencia estricta es crítica (por ejemplo, en reportes financieros o auditorías), y usar niveles menos restrictivos cuando el rendimiento es prioritario y un ligero riesgo de inconsistencia es aceptable.

